PRD: AI Short-Form Video Clipper

Product Requirements and Technical Design for a personal OpusClip-style tool that turns long videos into vertical short-form clips

Version: 1.0 | **Owner:** You | **Status:** Draft

1. Overview

1.1 Product summary

A tool that takes a long-form video (podcast, interview, webinar, YouTube video, stream) and automatically produces several ready-to-post vertical clips (9:16) for TikTok, Instagram Reels, and YouTube Shorts. It transcribes the video, finds the most engaging moments, cuts them, reframes them to vertical with the speaker kept in frame, adds animated captions, and exports the result.

1.2 Problem

Manually turning a 60-minute video into 8 to 10 good shorts takes hours: scrubbing the timeline, picking moments, cropping to vertical, writing captions, and exporting. Existing tools (OpusClip and others) work well but are subscription-priced, watermark free tiers, and limit customization.

1.3 Vision

A self-owned pipeline where you paste a link or upload a file, and 10 to 20 minutes later you have a set of scored, captioned, vertical clips you can edit lightly and publish.

1.4 Goals

- Generate 5 to 15 usable clips from a 30 to 90 minute video with minimal manual work.
- At least 60% of generated clips should be usable with no or light edits (your own quality bar).
- Processing time under roughly 0.3x of video duration on a GPU setup (a 60 min video in about 15 to 20 min).
- Low running cost per video (target under about $0.50 to $1 in compute and API costs).
- Full control over caption styles, branding, and clip length.

1.5 Non-goals (for v1)

- Full timeline-based video editor (like CapCut or Premiere).
- Multi-user teams, billing, and collaboration.
- Direct publishing to every platform (v1 exports files; publishing comes later).
- Live-stream clipping in real time.

2. Users and Use Cases

2.1 Primary user

You, as a solo creator or editor repurposing your own long-form content. The design keeps the door open for turning it into a multi-user product later.

2.2 Key user stories

ID	As a...	I want to...	So that...
US-1	Creator	paste a YouTube link or upload a video	I can start without manual file handling
US-2	Creator	get a list of suggested clips with scores	I can quickly pick the best ones
US-3	Creator	have clips auto-reframed to 9:16	the speaker is always visible
US-4	Creator	get animated word-by-word captions	clips hold attention without sound
US-5	Creator	edit the transcript and trim start/end	I can fix mistakes quickly
US-6	Creator	choose caption style, font, and colors	clips match my brand
US-7	Creator	download clips in 1080x1920 MP4	I can upload them anywhere
US-8	Creator	get a title, description, and hashtags per clip	posting is faster

3. Feature Scope

3.1 MVP (v1)

1. **Input:** upload a file (MP4, MOV, MKV, WebM) or paste a URL.
2. **Transcription:** speech-to-text with word-level timestamps.
3. **Clip detection:** LLM-based selection of 20 to 90 second self-contained moments.
4. **Virality score:** a 0 to 100 score with a short reason for each clip.
5. **Vertical reframing:** face/speaker tracking and a 9:16 crop.
6. **Captions:** animated word-highlight captions burned into the video.
7. **Export:** 1080x1920, H.264 MP4, loudness-normalized audio.
8. **Simple web UI:** project list, clip list with preview, and download.

3.2 v2

- Transcript-based editor (delete words to cut video).
- Multiple caption templates and brand kits (logo, fonts, colors, watermark).
- Speaker diarization, so the active speaker is followed in multi-person videos.
- Split-screen layout (for example, speaker on top and gameplay or screen share below).
- Auto B-roll or emoji and sound-effect suggestions.
- Title, description, and hashtag generation.
- Scheduling and publishing via platform APIs.
- Prompt-based clipping: "find moments where we talk about pricing".

3.3 v3 and later

- Multi-language transcription and translated captions plus AI dubbing.
- Performance feedback loop: learn from the views your published clips get.
- Team workspaces and billing.

4. User Flow

1. **Create project:** paste a link or upload a file.
2. **Processing:** the UI shows progress (downloading, transcribing, analyzing, rendering).
3. **Review:** a grid of clips, each with a thumbnail, duration, score, and the hook line.
4. **Edit (optional):** trim, change captions style, edit text, switch layout.

5. **Export:** download one clip or all clips; optionally copy the generated title and hashtags.

5. System Architecture

5.1 High-level diagram (text)

```
[Web UI] -> [API Server] -> [Job Queue] -> [Workers]
                |                              |
            [Postgres]          [Download] -> [Audio extract] -> [ASR]
                |               -> [Segmentation + LLM clip picking]
          [Object Storage]      -> [Face/speaker tracking + reframe]
                                -> [Caption render + FFmpeg export]
```

5.2 Components

Component	Responsibility
Frontend	Upload, project dashboard, clip review and editing, download
API server	Auth (optional for solo use), project CRUD, job creation, signed URLs, status endpoints
Job queue	Reliable async processing, retries, progress tracking
Workers	Run the pipeline stages; can be CPU or GPU nodes
Database	Projects, videos, transcripts, clips, jobs, render settings
Object storage	Original videos, extracted audio, intermediate files, final clips

5.3 Recommended stack

Layer	Recommendation	Notes
Frontend	Next.js (React) + Tailwind	Video preview with the HTML5 video element or Video.js

Layer	Recommendation	Notes
API	FastAPI (Python) or Node.js (NestJS/Express)	Python is convenient since the ML tooling is Python
Queue	Redis + Celery (Python) or BullMQ (Node)	Use separate queues for GPU and CPU tasks
Database	PostgreSQL	Use JSONB for transcripts and render settings
Storage	S3, Cloudflare R2, or MinIO locally	R2 has no egress fees, which suits video
Transcription	faster-whisper (local), Whisper large-v3, or a hosted API (Deepgram, AssemblyAI)	Need word-level timestamps
LLM	Claude API (or any strong LLM)	Clip selection, scoring, titles
Face tracking	MediaPipe, YOLOv8-face, or InsightFace	Per-frame face boxes
Rendering	FFmpeg plus ASS subtitles, or Remotion / Puppeteer for rich captions	See section 7
Downloading	yt-dlp	See legal notes in section 12
Deployment	Docker, with GPU workers on RunPod, Modal, Lambda Labs, or AWS	Local GPU works fine for personal use

6. Processing Pipeline (Detailed)

Each stage is an idempotent job that stores its output, so a failed run can resume from the last completed stage.

Stage 1: Ingestion

- **Upload:** use chunked or resumable upload (tus protocol or S3 multipart) since files can be several GB.
- **URL:** run yt-dlp to download the best video plus audio and merge them to MP4.

- **Validate:** run `ffprobe` to read duration, resolution, codec, frame rate, and audio streams. Reject files that are corrupt or exceed your length limit (for example, 3 hours).
- **Normalize:** transcode to a consistent working format if needed (H.264, AAC, constant frame rate) to avoid sync problems later.

Stage 2: Audio extraction

```
ffmpeg -i input.mp4 -vn -ac 1 -ar 16000 -c:a pcm_s16le audio.wav
```

Mono 16 kHz WAV is the standard input for Whisper-style models.

Stage 3: Transcription (ASR)

- Use faster-whisper with `word_timestamps=True` and VAD filtering to skip silence.
- Output a JSON structure of segments, each with words, start time, end time, and confidence.
- Optional: **diarization** (pyannote.audio or WhisperX) to label speakers.
- Optional: forced alignment (WhisperX) for more accurate word timings, which matters for caption sync.

```
{
  "language": "en",
  "segments": [
    {"start": 12.34, "end": 18.9, "speaker": "S1",
     "text": "...",
     "words": [{"w": "Here's", "s": 12.34, "e": 12.61}]}
  ]
}
```

Stage 4: Clip candidate generation

This is the core intelligence of the product. Use a two-pass approach.

Pass A: Segmentation. Split the transcript into topic blocks using sentence boundaries, long pauses (over about 1.5 s), speaker changes, and optionally embedding-based topic shifts (cosine similarity drops between sliding windows).

Pass B: LLM selection. Send the transcript (chunked with overlap if it exceeds the context window) with a prompt asking for clips that:

- have a strong hook in the first 3 seconds (question, bold claim, surprising fact, story start),
- are self-contained (make sense without the rest of the video),
- end at a natural conclusion or cliffhanger,
- run 20 to 90 seconds (configurable),
- do not begin or end mid-sentence.

Require **structured JSON output**:

```
{
  "clips": [
    {"start": 312.4, "end": 371.0,
     "hook": "The biggest mistake founders make is...",
     "title": "...",
     "score": 87,
     "score_breakdown": {"hook": 9, "flow": 8, "value": 9, "trend": 7},
     "reason": "Strong contrarian opening and clear payoff"}
  ]
}
```

Post-processing:

- Snap start and end times to the nearest word boundary, and extend slightly (about 0.2 to 0.4 s of padding) so words are not clipped.
- Remove overlapping clips (keep the higher score).
- Rank by score and keep the top N.

Scoring ideas. The LLM score is a start. You can blend it with signals you compute yourself:

Signal	How to compute
Speech energy and pace	Audio RMS and words per minute
Laughter and applause	Audio event classifier (YAMNet, PANNs)
Emotional intensity	Sentiment or emotion model on the transcript
Question and hook patterns	Rule-based plus LLM
Visual activity	Scene-change and motion detection (PySceneDetect)

Stage 5: Vertical reframing (9:16)

Target: 1080x1920 from a 16:9 source (for a 1920x1080 source, the crop window is about 608x1080, then scaled up).

1. **Scene detection:** PySceneDetect to find cuts, so the tracker resets at each shot.
2. **Face detection:** sample 5 to 10 frames per second inside each clip, running a detector for face boxes.
3. **Subject selection:** with multiple faces, choose the active speaker, using diarization timing matched to faces, or lip-motion detection (for example Light-ASD or TalkNet) for higher accuracy.
4. **Smooth the path:** apply a Kalman filter or exponential moving average to the crop center so the camera does not jitter, and limit the pan speed. Use a "dead zone" so small head movements do not move the frame.
5. **Layout decision per shot:**
   - One speaker: full-frame crop following the face.
   - Two speakers side by side: either follow the active speaker or use a stacked split (each face cropped to half the height).
   - Screen share or slides: show the full 16:9 frame letterboxed in the middle with a blurred copy of the video as background.
6. **Output:** a per-frame (or keyframed) crop rectangle `(x, y, w, h)` that feeds the render step.

Stage 6: Captions

- Group words into short caption chunks (1 to 4 words, max about 20 characters per line for readability).
- Highlight the currently spoken word (color change, scale pop, or underline).
- Generate styled subtitles as **ASS files** (supports per-word karaoke tags, outlines, shadows, and positioning), or render with a browser-based engine (Remotion, Puppeteer) for fancier animation.
- Place captions in the platform-safe zone: roughly the vertical middle to lower-middle, away from the bottom 15 to 20% where the UI covers content, and away from the right edge where the like and share buttons sit.
- Optional: emoji insertion per keyword, and automatic profanity masking.

Stage 7: Rendering and export

Specification for the final file:

Property	Value
Resolution	1080 x 1920 (9:16)
Frame rate	30 fps (or match source, max 60)
Video codec	H.264 (libx264) or NVENC `h264_nvenc` on GPU
Bitrate	CRF 18 to 22, or about 8 to 12 Mbps; `-preset medium` or faster
Pixel format	`yuv420p` (required for broad compatibility)
Audio codec	AAC, 128 to 192 kbps, 44.1 or 48 kHz
Loudness	Normalize to about -14 LUFS (`loudnorm` filter)
Container	MP4 with `-movflags +faststart`
Length limits	Up to 60 s is the safest for all platforms; Reels and TikTok allow longer

Example FFmpeg command (static crop, captions burned in):

```
ffmpeg -ss 312.4 -to 371.0 -i input.mp4 \
  -vf "crop=608:1080:656:0,scale=1080:1920,subtitles=captions.ass" \
  -af "loudnorm=I=-14:TP=-1.5:LRA=11" \
  -c:v libx264 -crf 20 -preset medium -pix_fmt yuv420p \
  -c:a aac -b:a 160k -movflags +faststart clip_01.mp4
```

For a **dynamic crop** that follows the speaker, generate a per-frame crop expression or process frames in Python (OpenCV or PyAV), pipe them to FFmpeg, then mux the original audio. For the **blurred-background layout**, use a filter graph that splits the input, scales and blurs one copy to fill 1080x1920, and overlays the sharp 16:9 copy in the center.

Stage 8: Metadata generation

For each clip, use the LLM to generate:

- 3 title options (hook-style, under 60 characters),
- a short description,

- 5 to 10 hashtags,
- a suggested thumbnail frame (highest face-visibility and expression score).

7. Data Model

Table	Key fields
`projects`	id, name, source_type (upload/url), source_url, status, created_at
`videos`	id, project_id, storage_key, duration, width, height, fps, codec, language
`transcripts`	id, video_id, json (JSONB with words and segments), model, created_at
`clips`	id, project_id, start, end, score, score_breakdown (JSONB), hook, title, description, hashtags, status, selected
`render_settings`	id, clip_id, layout, caption_style (JSONB), crop_keyframes (JSONB), brand_kit_id
`renders`	id, clip_id, storage_key, resolution, size_bytes, created_at
`jobs`	id, project_id, stage, status, progress, error, attempts, started_at, finished_at
`brand_kits`	id, name, logo_key, fonts, colors, default_caption_style (JSONB)

8. API Design (REST)

Method	Endpoint	Purpose
POST	`/api/projects`	Create a project from a URL, or start an upload
POST	`/api/uploads/sign`	Get presigned multipart upload URLs
GET	`/api/projects/{id}`	Project status, progress, and stage

Method	Endpoint	Purpose
GET	`/api/projects/{id}/clips`	List the generated clips
PATCH	`/api/clips/{id}`	Update trim points, title, caption text, or style
POST	`/api/clips/{id}/render`	Re-render a clip with new settings
GET	`/api/clips/{id}/download`	Signed URL for the final MP4
DELETE	`/api/projects/{id}`	Delete the project and its stored files

Use **WebSockets or Server-Sent Events** for live progress, or poll `GET /projects/{id}` every few seconds.

9. Non-Functional Requirements

Area	Requirement
Performance	60 min video processed in under about 20 min on one mid-range GPU; rendering parallelized across clips
Scalability	Stateless workers; scale by adding queue consumers; separate GPU and CPU queues
Reliability	Idempotent stages, retries with backoff (3 attempts), resume from the last completed stage
Storage	Lifecycle rules: delete originals and intermediates after N days; keep final clips
Security	Presigned URLs with expiry, file type and size validation, sandboxed FFmpeg workers, no arbitrary command injection from URLs or filenames
Privacy	Option to process fully locally (local Whisper plus a local LLM) so content never leaves your machine
Observability	Structured logs, per-stage timing metrics, error tracking (Sentry)

Area	Requirement
Accessibility	Captions are burned in, and an SRT or VTT file is also exported

10. Infrastructure and Cost Estimate

Rough numbers for a 60-minute video (verify against current pricing, since these change):

Item	Approach	Approx. cost
Transcription	Local faster-whisper on your GPU	Free (electricity)
Transcription	Hosted API	Roughly $0.15 to $0.50 per hour of audio
LLM clip selection	About 15k to 25k input tokens plus a small output	Typically a few cents to under $0.50, depending on the model
Face tracking	Local CPU/GPU	Free
Rendering	FFmpeg on CPU, or NVENC on GPU	Free locally; cents on cloud GPUs
Storage	R2/S3 for a few GB per project	Cents per month

Personal-use setup: one machine with an NVIDIA GPU (8 GB VRAM or more is comfortable for Whisper large-v3 via faster-whisper), Docker Compose running Postgres, Redis, MinIO, the API, and a worker. Move to cloud GPUs only if you need to scale.

11. Success Metrics

Metric	Target
Clip acceptance rate (clips you keep without edits)	60% or more
Time from upload to first clips	Under 20 min for 60 min video
Caption timing error	Under 100 ms
Face-in-frame rate in reframed clips	95% or more

Metric	Target
Pipeline failure rate	Under 2% of jobs
Cost per processed hour of video	Under about $1

12. Legal, Ethical, and Platform Considerations

- **Copyright:** only clip content you own or have permission to repurpose. Clipping other people's videos can lead to takedowns and strikes.
- **Downloading from YouTube and others:** downloading with yt-dlp may violate the platform's Terms of Service. For safety, use it on your own content, or prefer direct file uploads.
- **Third-party APIs:** posting automatically through TikTok, Instagram, and YouTube APIs requires app review and OAuth, and each has its own rate limits and rules.
- **Voices and faces:** if you later add dubbing or face-related features, consider consent and disclosure rules.
- **Data retention:** if this ever serves other users, add a privacy policy, deletion controls, and clear retention periods.

13. Risks and Mitigations

Risk	Impact	Mitigation
LLM picks weak or out-of-context clips	Low quality output	Add a second-pass "critic" prompt, blend with audio and engagement signals, allow prompt-based clipping
Wrong face tracked in multi-person videos	Speaker cut out of frame	Use diarization plus active-speaker detection; fall back to a split or blurred layout
Caption drift versus audio	Looks unprofessional	Use forced alignment (WhisperX); validate timing in tests
Cuts mid-sentence	Awkward clips	Snap to word boundaries and sentence ends; add padding

Risk	Impact	Mitigation
Large files break uploads	Frustrating UX	Resumable multipart uploads
Long processing time	Slow feedback	Parallelize rendering per clip; show progress and early results
Hallucinated timestamps from the LLM	Clips at the wrong place	Give the LLM numbered, timestamped lines, and validate that returned times exist in the transcript

14. Build Plan and Milestones

Phase	Scope	Rough effort
0. Prototype	A single Python script: video -> Whisper -> LLM picks clips -> FFmpeg center-crop with ASS captions	2 to 4 days
1. Pipeline	Queue and workers, database, storage, stage-by-stage resumability	1 to 2 weeks
2. Smart reframing	Face detection, smoothing, scene detection, layout switching	1 to 2 weeks
3. Web UI	Upload, progress, clip grid, preview, download	1 to 2 weeks
4. Editing	Trim handles, transcript editing, caption style presets, re-render	1 to 2 weeks
5. Polish	Metadata generation, brand kits, scheduling and publishing	2+ weeks

Tip: do Phase 0 first. A command-line prototype will show you within a few days whether clip quality is good enough and where the real difficulty lies (usually reframing and clip selection quality, not the UI).

15. Suggested Repository Structure

```
clipper/
├── apps/
│   ├── web/                # Next.js frontend
│   └── api/                # FastAPI backend
├── workers/
│   ├── ingest.py           # download, validate, normalize
│   ├── transcribe.py       # faster-whisper / WhisperX
│   ├── select_clips.py     # segmentation + LLM prompts
│   ├── reframe.py          # face tracking, crop paths
│   ├── captions.py         # ASS generation / caption render
│   └── render.py           # FFmpeg composition and export
├── prompts/                # versioned LLM prompt templates
├── infra/                  # docker-compose, Dockerfiles, IaC
└── tests/                  # sample videos + golden outputs
```

16. Open Questions

1. Will this stay personal, or become a product for other users? (Affects auth, billing, and legal work.)
2. What content types matter most: talking-head podcasts, gaming, tutorials, or webinars? (Affects reframing logic.)
3. Run fully local, or use cloud APIs for transcription and the LLM?
4. Which languages must be supported?
5. Do you want automatic publishing, or is exporting files enough?
6. What is the maximum video length you plan to process?

This document is a starting point. Tools, pricing, and platform rules change often, so verify current details before committing to any specific vendor or API.